

Greedy Algorithms

Sean

May 16, 2026



Outline

- The greedy idea
- How to justify greedy
- Common greedy patterns
- Implementation habits

Introduction

- A greedy algorithm builds an answer by repeatedly making the best local choice



Introduction

- A greedy algorithm builds an answer by repeatedly making the best local choice
- The hard part is not usually coding the choice



Introduction

- A greedy algorithm builds an answer by repeatedly making the best local choice
- The hard part is not usually coding the choice
- The hard part is proving that the local choice cannot hurt the final answer



1 The Greedy Idea

2 How to Justify Greedy

3 Common Greedy Patterns

4 Implementation Habits



The Greedy Idea

What Makes an Algorithm Greedy?

- Make one choice that looks best right now



What Makes an Algorithm Greedy?

- Make one choice that looks best right now
- Commit to it permanently



What Makes an Algorithm Greedy?

- Make one choice that looks best right now
- Commit to it permanently
- Reduce the remaining problem



What Makes an Algorithm Greedy?

- Make one choice that looks best right now
- Commit to it permanently
- Reduce the remaining problem
- Repeat until the answer is complete



Greedy Is Not Guessing

- A greedy rule must be tied to the objective



Greedy Is Not Guessing

- A greedy rule must be tied to the objective
- “Take the largest” and “take the smallest” are not proofs



Greedy Is Not Guessing

- A greedy rule must be tied to the objective
- “Take the largest” and “take the smallest” are not proofs
- We need a reason that any optimal answer can be changed to include our choice



Greedy Is Not Guessing

- A greedy rule must be tied to the objective
- “Take the largest” and “take the smallest” are not proofs
- We need a reason that any optimal answer can be changed to include our choice
- If that reason is false, the greedy solution may fail on a small counterexample



Example: Coin Change Counterexample

Toy task

Given coin values, choose coins to reach a target using as few coins as possible.

Example: Coin Change Counterexample

Toy task

Given coin values, choose coins to reach a target using as few coins as possible.

- For coin values 1, 5, 10, 25, taking the largest possible coin works



Example: Coin Change Counterexample

Toy task

Given coin values, choose coins to reach a target using as few coins as possible.

- For coin values 1, 5, 10, 25, taking the largest possible coin works
- For coin values 1, 3, 4 and target 6, it fails



Example: Coin Change Counterexample

Toy task

Given coin values, choose coins to reach a target using as few coins as possible.

- For coin values 1, 5, 10, 25, taking the largest possible coin works
- For coin values 1, 3, 4 and target 6, it fails
- Greedy needs a structural reason, not just an intuitive rule



When Greedy Often Appears

- We choose items, intervals, tasks, or events one at a time

When Greedy Often Appears

- We choose items, intervals, tasks, or events one at a time
- Earlier choices constrain what can happen later

When Greedy Often Appears

- We choose items, intervals, tasks, or events one at a time
- Earlier choices constrain what can happen later
- Sorting reveals a natural order to make decisions



When Greedy Often Appears

- We choose items, intervals, tasks, or events one at a time
- Earlier choices constrain what can happen later
- Sorting reveals a natural order to make decisions
- A data structure tracks the best currently available option



A Greedy Workflow

- Observe: guess a rule suggested by the problem structure



A Greedy Workflow

- Observe: guess a rule suggested by the problem structure
- Test: try to break that rule on small cases



A Greedy Workflow

- Observe: guess a rule suggested by the problem structure
- Test: try to break that rule on small cases
- Verify: prove why the rule is safe



Step 1: Observe

- Ask what local choice seems to support the global objective



Step 1: Observe

- Ask what local choice seems to support the global objective
- Look for a useful order: earliest, smallest, largest, or currently best



Step 1: Observe

- Ask what local choice seems to support the global objective
- Look for a useful order: earliest, smallest, largest, or currently best
- Treat this as a candidate rule, not yet a solution



Step 2: Test

- Try tiny examples before trusting the rule

Step 2: Test

- Try tiny examples before trusting the rule
- Search for ties, extremes, and choices that block later options

Step 2: Test

- Try tiny examples before trusting the rule
- Search for ties, extremes, and choices that block later options
- One counterexample is enough to reject the rule

Step 3: Verify

- State exactly what choice becomes permanent



Step 3: Verify

- State exactly what choice becomes permanent
- Prove that some optimal solution can include that choice

Step 3: Verify

- State exactly what choice becomes permanent
- Prove that some optimal solution can include that choice
- Show the remaining problem still has the same form



1 The Greedy Idea

2 How to Justify Greedy

3 Common Greedy Patterns

4 Implementation Habits



How to Justify Greedy

Proof Toolkit

- Contradiction: assume a better solution exists, then derive an impossibility



Proof Toolkit

- Contradiction: assume a better solution exists, then derive an impossibility
- Induction: prove the greedy claim remains true after every step



Proof Toolkit

- Contradiction: assume a better solution exists, then derive an impossibility
- Induction: prove the greedy claim remains true after every step
- Constructive proof: build or transform a valid solution directly



How They Show Up in Greedy

- Contradiction: assume a schedule beats the greedy one



How They Show Up in Greedy

- Contradiction: assume a schedule beats the greedy one
- Induction: after each choice, greedy is still no worse



How They Show Up in Greedy

- Contradiction: assume a schedule beats the greedy one
- Induction: after each choice, greedy is still no worse
- Constructive proof: exchange one optimal choice for the greedy choice



The Greedy Choice Property

- At least one optimal solution starts with the greedy choice



The Greedy Choice Property

- At least one optimal solution starts with the greedy choice
- After making that choice, the remaining problem has the same form



The Greedy Choice Property

- At least one optimal solution starts with the greedy choice
- After making that choice, the remaining problem has the same form
- This lets us repeat the same argument for the next choice



Exchange Argument

- Start with an arbitrary optimal solution

Exchange Argument

- Start with an arbitrary optimal solution
- If it already contains the greedy choice, we are done



Exchange Argument

- Start with an arbitrary optimal solution
- If it already contains the greedy choice, we are done
- Otherwise, exchange part of it with the greedy choice



Exchange Argument

- Start with an arbitrary optimal solution
- If it already contains the greedy choice, we are done
- Otherwise, exchange part of it with the greedy choice
- Show the new solution is still valid and no worse



Staying Ahead

- Compare the greedy solution with any other valid solution



Staying Ahead

- Compare the greedy solution with any other valid solution
- After each step, prove greedy is at least as good by a useful measure



Staying Ahead

- Compare the greedy solution with any other valid solution
- After each step, prove greedy is at least as good by a useful measure
- Common measures: earliest finish time, most remaining capacity, smallest cost so far



Staying Ahead

- Compare the greedy solution with any other valid solution
- After each step, prove greedy is at least as good by a useful measure
- Common measures: earliest finish time, most remaining capacity, smallest cost so far
- At the end, staying ahead implies optimality



- 1 The Greedy Idea
- 2 How to Justify Greedy
- 3 Common Greedy Patterns
- 4 Implementation Habits



Common Greedy Patterns

Pattern: Sort by Earliest Finish

- We want to choose as many non-overlapping intervals as possible

Pattern: Sort by Earliest Finish

- We want to choose as many non-overlapping intervals as possible
- Choose the interval that finishes earliest

Pattern: Sort by Earliest Finish

- We want to choose as many non-overlapping intervals as possible
- Choose the interval that finishes earliest
- Skip every interval that starts before it ends

Pattern: Sort by Earliest Finish

- We want to choose as many non-overlapping intervals as possible
- Choose the interval that finishes earliest
- Skip every interval that starts before it ends
- Finishing early leaves the most room for future choices



Why Not Other Rules?

Earliest start can choose one long interval



Shortest duration can block two intervals



Example: Movie Festival

CSES: Movie Festival

Given starting and ending times of movies, watch as many complete movies as possible.

Example: Movie Festival

CSES: Movie Festival

Given starting and ending times of movies, watch as many complete movies as possible.

- Sort movies by ending time

Example: Movie Festival

CSES: Movie Festival

Given starting and ending times of movies, watch as many complete movies as possible.

- Sort movies by ending time
- Take a movie if its start time is at least the end time of the last chosen movie



Example: Movie Festival

CSES: Movie Festival

Given starting and ending times of movies, watch as many complete movies as possible.

- Sort movies by ending time
- Take a movie if its start time is at least the end time of the last chosen movie
- Exchange argument: any first movie ending later can be replaced by the earliest-ending movie



Pattern: Sort and Pair Extremes

- Sort values so smallest and largest choices are visible



Pattern: Sort and Pair Extremes

- Sort values so smallest and largest choices are visible
- Pair the largest item with the smallest item that can fit with it



Pattern: Sort and Pair Extremes

- Sort values so smallest and largest choices are visible
- Pair the largest item with the smallest item that can fit with it
- If the smallest cannot fit, no other remaining item can fit with the largest



Pattern: Sort and Pair Extremes

- Sort values so smallest and largest choices are visible
- Pair the largest item with the smallest item that can fit with it
- If the smallest cannot fit, no other remaining item can fit with the largest
- This usually leads to a two-pointer implementation



Example: Ferris Wheel

CSES: Ferris Wheel

Each gondola holds at most two children and has weight limit x . Minimize the number of gondolas.



Example: Ferris Wheel

CSES: Ferris Wheel

Each gondola holds at most two children and has weight limit x . Minimize the number of gondolas.

- Sort all weights

Example: Ferris Wheel

CSES: Ferris Wheel

Each gondola holds at most two children and has weight limit x . Minimize the number of gondolas.

- Sort all weights
- Put the heaviest remaining child into a gondola

Example: Ferris Wheel

CSES: Ferris Wheel

Each gondola holds at most two children and has weight limit x . Minimize the number of gondolas.

- Sort all weights
- Put the heaviest remaining child into a gondola
- If the lightest remaining child fits with them, pair them



Example: Ferris Wheel

CSES: Ferris Wheel

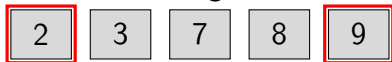
Each gondola holds at most two children and has weight limit x . Minimize the number of gondolas.

- Sort all weights
- Put the heaviest remaining child into a gondola
- If the lightest remaining child fits with them, pair them
- Otherwise, the heaviest child must ride alone



Trace: Ferris Wheel

$x = 10$, sorted weights



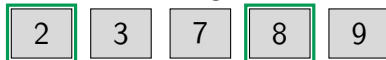
$2 + 9 > 10$, so 9 rides alone

Gondolas so far



Trace: Ferris Wheel

$x = 10$, sorted weights



$2 + 8 \leq 10$, so pair them

Gondolas so far



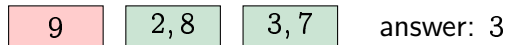
Trace: Ferris Wheel

$x = 10$, sorted weights



$3 + 7 \leq 10$, so pair them

Gondolas so far



Pattern: Keep the Best Available

- Process events in an order where decisions become available over time

Pattern: Keep the Best Available

- Process events in an order where decisions become available over time
- Store available choices in a priority queue



Pattern: Keep the Best Available

- Process events in an order where decisions become available over time
- Store available choices in a priority queue
- Whenever we must decide, take the best currently available option



Pattern: Keep the Best Available

- Process events in an order where decisions become available over time
- Store available choices in a priority queue
- Whenever we must decide, take the best currently available option
- This is common in scheduling and resource allocation



Example: Tasks and Deadlines

Classic task

Each task takes one day, has a deadline, and gives a reward.
Complete at most one task per day. Maximize total reward.

Example: Tasks and Deadlines

Classic task

Each task takes one day, has a deadline, and gives a reward. Complete at most one task per day. Maximize total reward.

- Process days from late to early

Example: Tasks and Deadlines

Classic task

Each task takes one day, has a deadline, and gives a reward. Complete at most one task per day. Maximize total reward.

- Process days from late to early
- On each day, add all unconsidered tasks with deadline at least today



Example: Tasks and Deadlines

Classic task

Each task takes one day, has a deadline, and gives a reward. Complete at most one task per day. Maximize total reward.

- Process days from late to early
- On each day, add all unconsidered tasks with deadline at least today
- Use `priority_queue` to choose the best reward for this day



Trace: Tasks and Deadlines

Tasks: $A(3, 40)$, $B(3, 30)$, $C(2, 70)$, $D(1, 10)$

Process days from late to early

40

Day 3: available rewards 40, 30

Choose 40

Trace: Tasks and Deadlines

Tasks: $A(3, 40)$, $B(3, 30)$, $C(2, 70)$, $D(1, 10)$

Process days from late to early

40

70

Day 2: add reward 70

Choose 70

Trace: Tasks and Deadlines

Tasks: $A(3, 40)$, $B(3, 30)$, $C(2, 70)$, $D(1, 10)$

Process days from late to early

40

70

Day 1: add reward 10

30

total: 140

Choose 30

- 1 The Greedy Idea
- 2 How to Justify Greedy
- 3 Common Greedy Patterns
- 4 Implementation Habits



Implementation Habits

Interval Scheduling Code

```
1  vector<pair<int, int>> movies; // {end, start}
2  sort(movies.begin(), movies.end());
3
4  int ans = 0;
5  int last_end = 0;
6  for (auto [end, start] : movies) {
7      if (start >= last_end) {
8          ans++;
9          last_end = end;
10     }
11 }
```



Two-Pointer Greedy Code

```
1  sort(w.begin(), w.end());
2
3  int l = 0, r = n - 1;
4  int ans = 0;
5  while (l <= r) {
6      if (w[l] + w[r] <= x) l++;
7      r--;
8      ans++;
9  }
```



Priority Queue Greedy Code

```
1  vector<pair<int, int>> tasks; // {deadline, reward}
2  sort(tasks.rbegin(), tasks.rend());
3  priority_queue<int> pq;
4  long long ans = 0;
5  int i = 0;
6  for (int day = max_deadline; day >= 1; day--) {
7      while (i < n && tasks[i].first >= day) {
8          pq.push(tasks[i++].second);
9      }
10     if (pq.empty()) continue;
11     ans += pq.top();
12     pq.pop();
13 }
```



Greedy Checklist

- Sort by the key that matches the proof
- Decide how ties should behave
- State what choice is permanent
- Prove the choice is safe with an exchange or staying-ahead argument
- Test small counterexamples before coding



Takeaways

- Greedy commits to the best local choice
- The proof matters more than the implementation trick
- Sorting often reveals the order in which choices become safe
- Priority queues help when the best available choice changes over time



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

